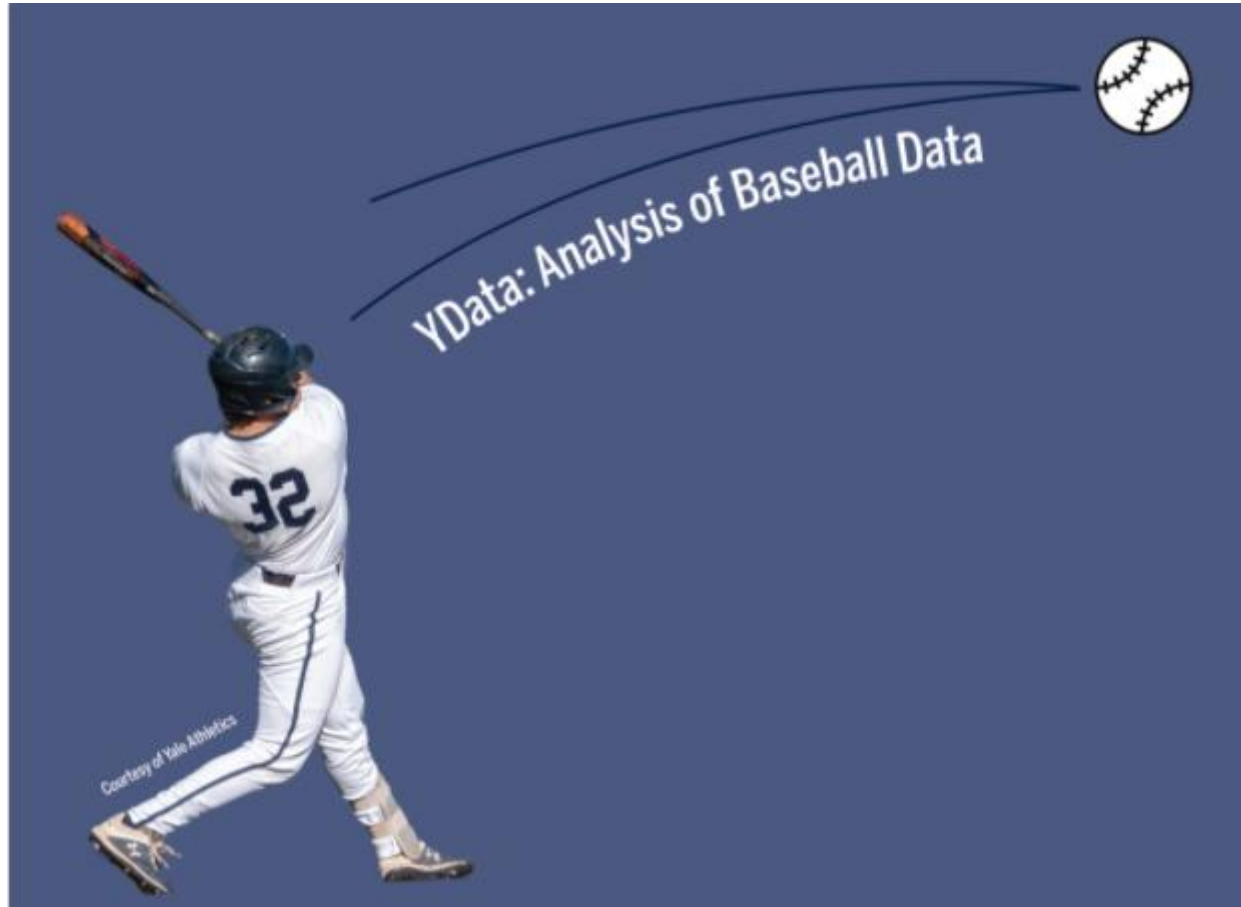


S&DS 1730



Class 3: Data manipulation with pandas

Overview

Brief discussion of preface and prologue to Astroball

Writing functions in Python

Transforming/manipulating baseball data using Pandas

If there is time

- Work on the labs
- Assessing what impressive statistics are using percentiles, boxplots, and histograms

Announcement: Lab 1 and Lab 2

Lab 1 has been released. It is due on **Wednesday January 28th**

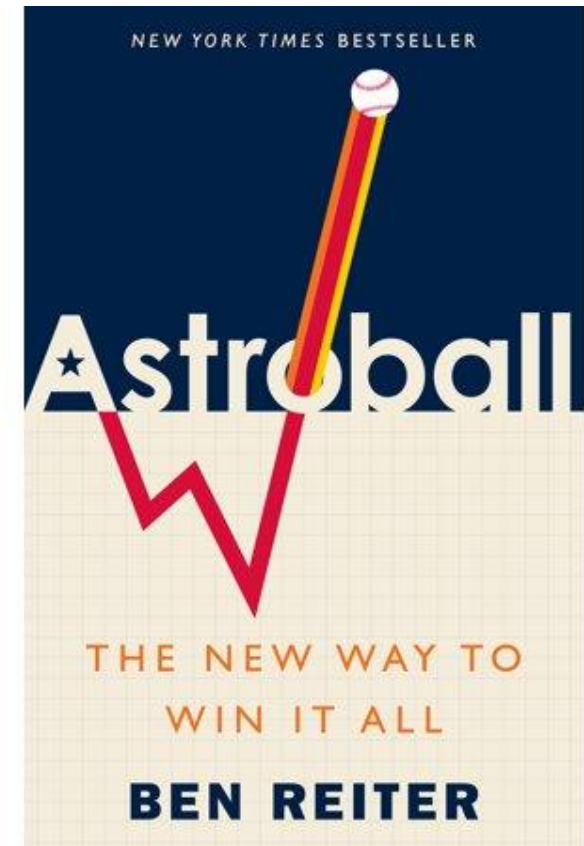
Lab 2 has also been released. It is due on **Sunday February 1st**

How is lab 1 going?

Discussion on chapter 1 of Astrobball

Let's discuss the chapter for 3 minutes in small groups of 3-4 people and then have a larger conversation as a group

- E.g., Discuss your quote and reaction to chapter 1



Discussion on chapter 1 of Astrobball

Interesting quotes and reactions to share?



Sig Mejdal



Jeff Luhnow

Discussion on chapter 1 of Astrobball

Structure of the draft:

- 50 rounds, ~1,500 players drafted (30 teams * 50 rounds)

Pg. 30: A study in 2013 found that:

1. “if a club’s draft produces nine players who appear in the majors for even a single game, it’s in the 95th percentile.”
2. “The 95th percentile for number of everyday players (def 1,500 PA) is four.”

Discussion on chapter 1 of Astrobball

More information on the 2009 draft:

- [Wikipedia](#)
- [Discussion of the Cardinals' picks](#)
- [Wikipedia information on baseball drafts in general](#)
- [Full draft results](#)



Joe Kelly

Round 3, 89th pick



Matt Carpenter

Round 13, 399th pick



Trevor Rosenthal

Round 21, 639th pick

Last class

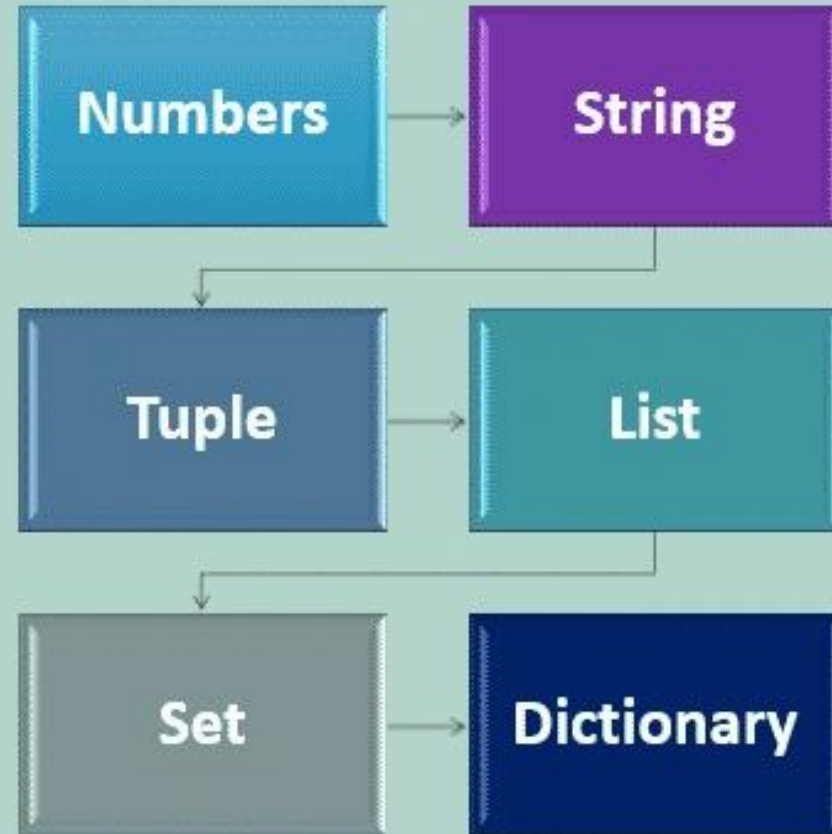
Last class we watched an inning of the 2025 All-Star game and discussed the retrosheet play-by-play data



	gid	inning	top_bot	outs_pre	score_v	score_h	batteam	batter	pitcher	pitches	event
0	NLS202507150	1	0	0	0	0	ALS	torrg001	skenp001	FBTBS	K
1	NLS202507150	1	0	1	0	0	ALS	greer003	skenp001	BCCBFBS	K
2	NLS202507150	1	0	2	0	0	ALS	judga001	skenp001	CX	43/G4M
3	NLS202507150	1	1	0	0	0	NLS	ohtas001	skubt001	FFBX	S8/L8
4	NLS202507150	1	1	0	0	0	NLS	acunr001	skubt001	FSBX	S4/G4S-.1-2
5	NLS202507150	1	1	0	0	0	NLS	martk001	skubt001	SX	D9/G3+.2-H;1-H
6	NLS202507150	1	1	0	0	2	NLS	freef001	skubt001	BSX	53/G56
7	NLS202507150	1	1	1	0	2	NLS	machm001	skubt001	FSC	K
8	NLS202507150	1	1	2	0	2	NLS	smitw003	skubt001	CTS	K
9	NLS202507150	2	0	0	0	2	ALS	ralec001	kersc001	.X	7/L7LD+

Python

Data Types



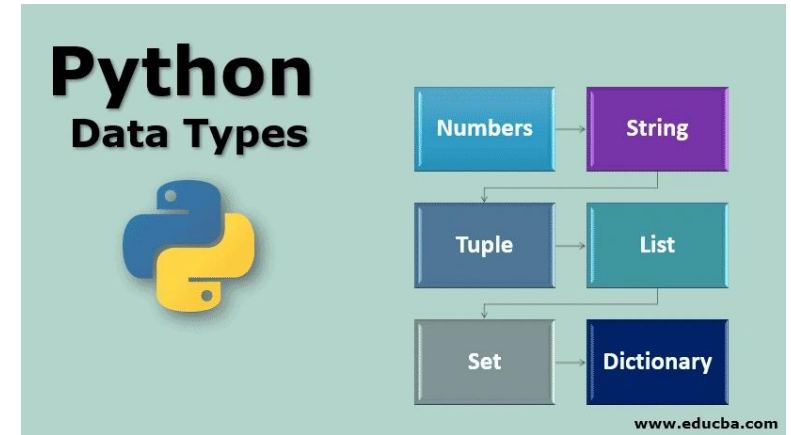
Last class

We discussed the basics of Python

- Data types: ints, floats, strings and Booleans
- Data structures: lists [], tuples (), and dictionaries {}
- For loops
- Array computations with numpy, and Boolean indexing

```
my_array = np.array([12, 4, 3])
```

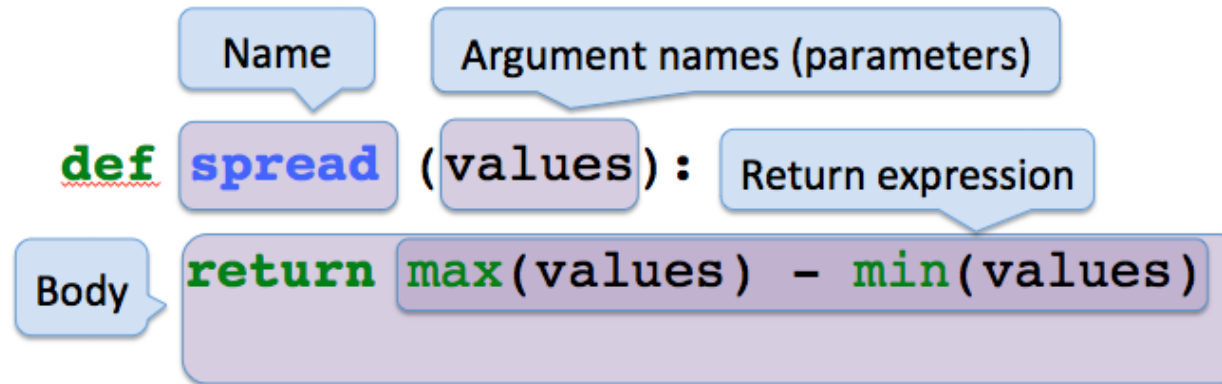
```
my_array[my_array < 5]
```



Any questions?

Writing functions

User-defined functions give names to blocks of code



Functions can return multiple values by returning a tuples of values

- We can use "tuple unpacking" to assign each value to its own variable

Let's explore this in Jupyter!

Pandas DataFrames



Pandas: Series and DataFrames

[Pandas](#) is a package that is useful for processing time series and tables of data

There are two main data structures in pandas:

- **Series:** represent one-dimensional data
- **DataFrames:** represent data tables
 - i.e., relational data



pandas Series

Series are one-dimensional ndarrays with index names

yearID	
2021	46
2022	34
2023	44
2024	54

Index

array of values

Often, we treat Series similar to numpy ndarrays

- E.g., we do things like: `np.sum(my_series)`

pandas DataFrames

Pandas DataFrame hold Table data

This is one of the most useful formats to extract insights from datasets

Pandas DataFrames have an Index, and columns of data

The Index here is just sequential numbers but we could set it to be the playerId or values in another column



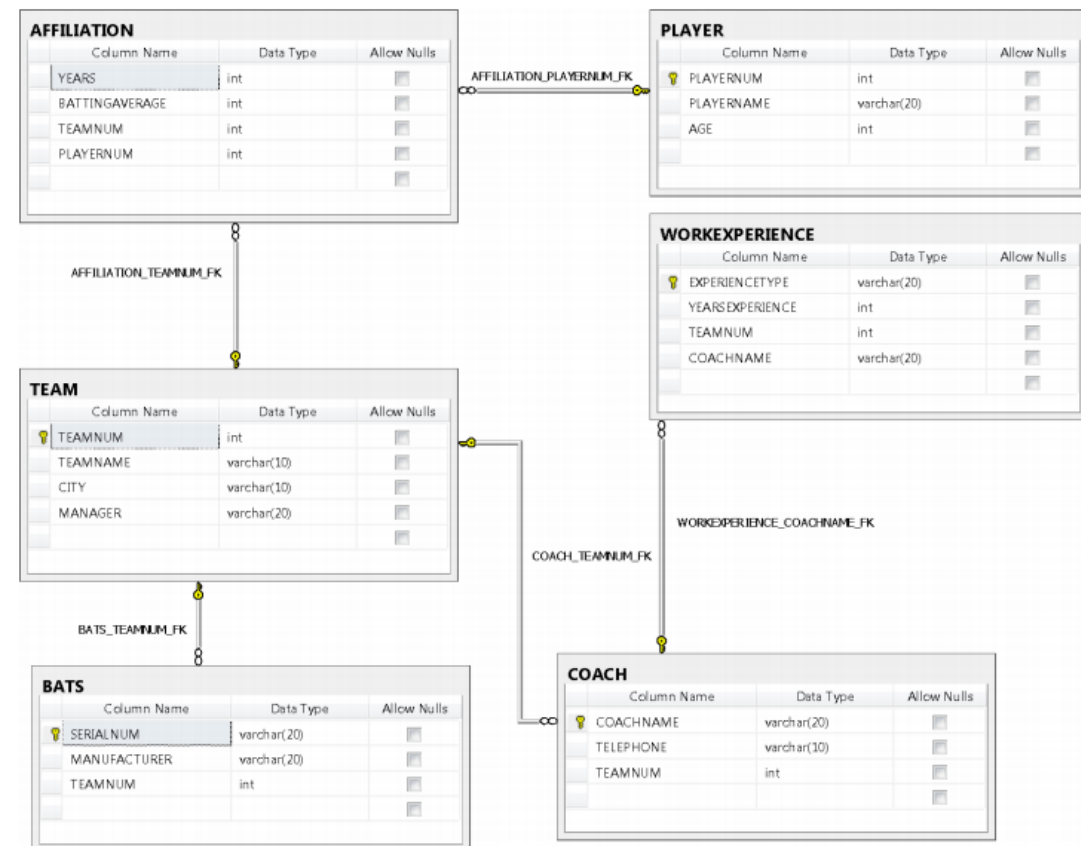
	playerID	yearID	stint	teamID	lgID	G	G_batting	AB	R	H
1	aardsda01	2004	1	SFN	NL	11	11	0	0	0
2	aardsda01	2006	1	CHN	NL	45	43	2	0	0
3	aardsda01	2007	1	CHA	AL	25	2	0	0	0
4	aardsda01	2008	1	BOS	AL	47	5	1	0	0
5	aardsda01	2009	1	SEA	AL	73	3	0	0	0
6	aardsda01	2010	1	SEA	AL	53	4	0	0	0
7	aardsda01	2012	1	NYA	AL	1	NA	NA	NA	NA
8	aaronha01	1954	1	ML1	NL	122	122	468	58	131

Lahman Database: Season level statistics

Lahman database contains season level information about Major League baseball players from 1871 to 2024

```
import pandas as pd
```

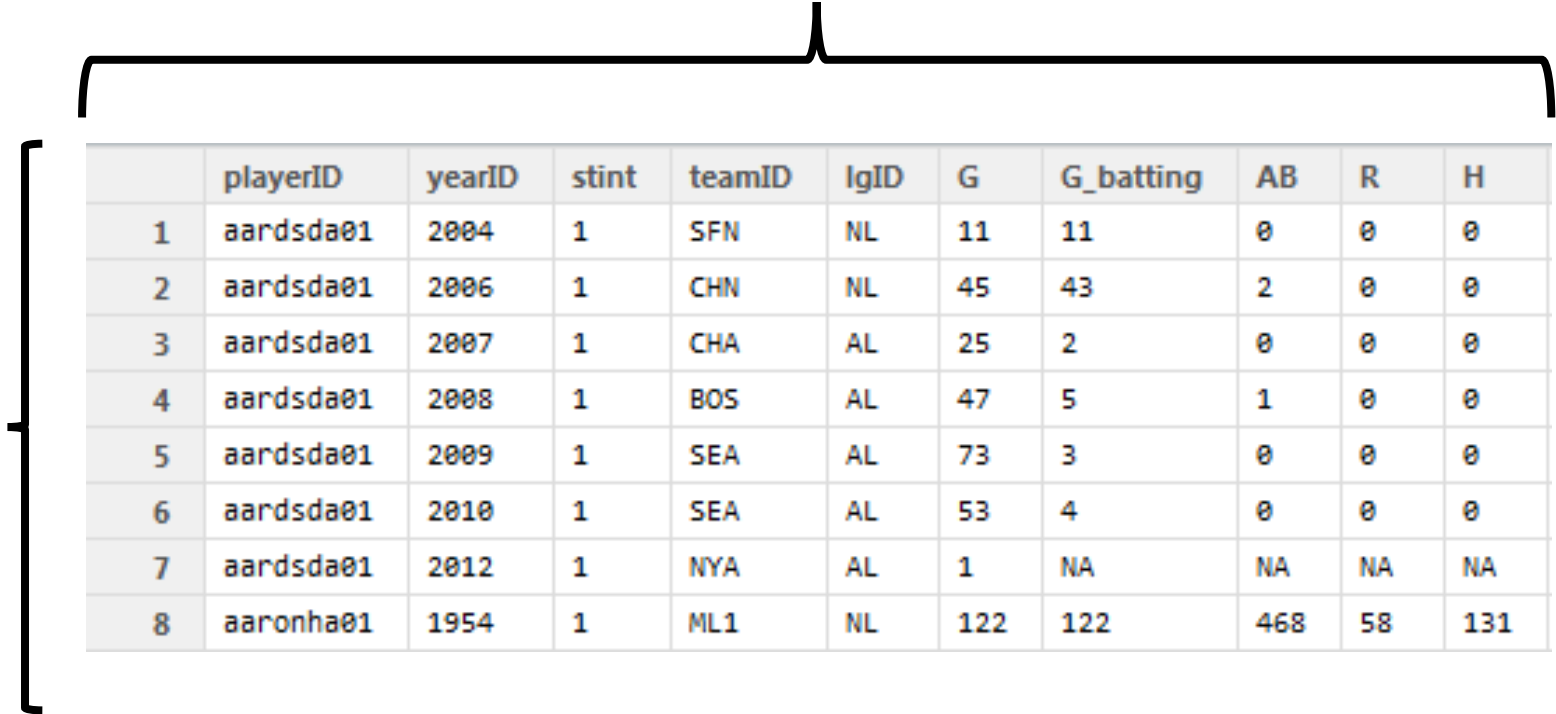
```
batting = pd.read_csv('Batting.csv')
```



Structured data: Batting dataset

Variables

Cases



	playerID	yearID	stint	teamID	lgID	G	G_batting	AB	R	H
1	aardsda01	2004	1	SFN	NL	11	11	0	0	0
2	aardsda01	2006	1	CHN	NL	45	43	2	0	0
3	aardsda01	2007	1	CHA	AL	25	2	0	0	0
4	aardsda01	2008	1	BOS	AL	47	5	1	0	0
5	aardsda01	2009	1	SEA	AL	73	3	0	0	0
6	aardsda01	2010	1	SEA	AL	53	4	0	0	0
7	aardsda01	2012	1	NYA	AL	1	NA	NA	NA	NA
8	aaronha01	1954	1	ML1	NL	122	122	468	58	131

Structured data

Variables

Cases

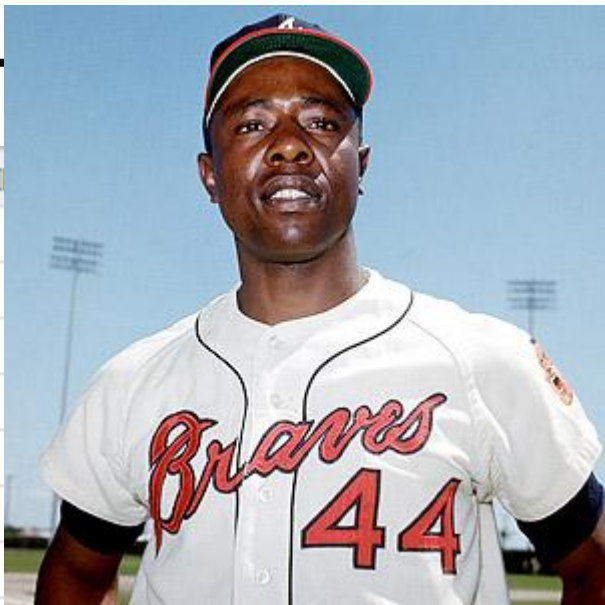
	playerID	yearID	stint	teamID	lgID	G	G_batting	AB	R	H
1	aardsda01	2004	1					0	0	0
2	aardsda01	2006	1					2	0	0
3	aardsda01	2007	1					0	0	0
4	aardsda01	2008	1					1	0	0
5	aardsda01	2009	1					0	0	0
6	aardsda01	2010	1					0	0	0
7	aardsda01	2012	1					NA	NA	NA
8	aaronha01	1954	1	ML1	NL	122	122	468	58	131

Structured data

Variables

Cases

	playerID	yearID	stint	teamID	lgID					
1	aardsda01	2004	1	SFN	NL					
2	aardsda01	2006	1	CHN	NL					
3	aardsda01	2007	1	CHA	AL					
4	aardsda01	2008	1	BOS	AL					
5	aardsda01	2009	1	SEA	AL					
6	aardsda01	2010	1	SEA	AL					
7	aardsda01	2012	1	NYA	AL					
8	aaronha01	1954	1	ML1	NL	122	122	468	58	131



Categorical and Quantitative Variables

Categorical Variable

Quantitative Variable

Cases

	playerID	yearID	stint	teamID	lgID	G	G_batting	AB	R	H
1	aardsda01	2004	1	SFN	NL	11	11	0	0	0
2	aardsda01	2006	1	CHN	NL	45	43	2	0	0
3	aardsda01	2007	1	CHA	AL	25	2	0	0	0
4	aardsda01	2008	1	BOS	AL	47	5	1	0	0
5	aardsda01	2009	1	SEA	AL	73	3	0	0	0
6	aardsda01	2010	1	SEA	AL	53	4	0	0	0
7	aardsda01	2012	1	NYA	AL	1	NA	NA	NA	NA
8	aaronha01	1954	1	ML1	NL	122	122	468	58	131

Explanatory and Response Variables

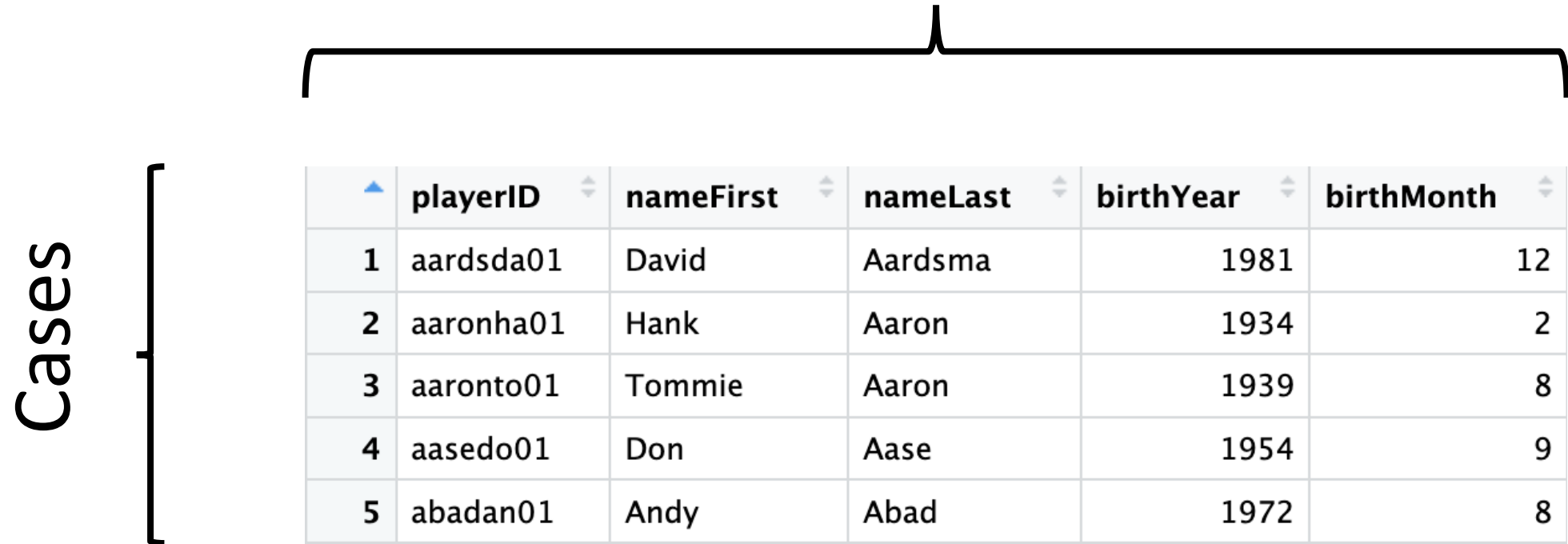
Sometimes we use one variable (**the explanatory variable**) to understand/predict another variable (**the response variable**)

	playerID	yearID	stint	teamID	lgID	G	G_batting	AB	R	H
1	aardsda01	2004	1	SFN	NL	11	11	0	0	0
2	aardsda01	2006	1	CHN	NL	45	43	2	0	0
3	aardsda01	2007	1	CHA	AL	25	2	0	0	0
4	aardsda01	2008	1	BOS	AL	47	5	1	0	0
5	aardsda01	2009	1	SEA	AL	73	3	0	0	0
6	aardsda01	2010	1	SEA	AL	53	4	0	0	0
7	aardsda01	2012	1	NYA	AL	1	NA	NA	NA	NA
8	aaronha01	1954	1	ML1	NL	122	122	468	58	131

People Dataset: Information on players

Variables

Cases

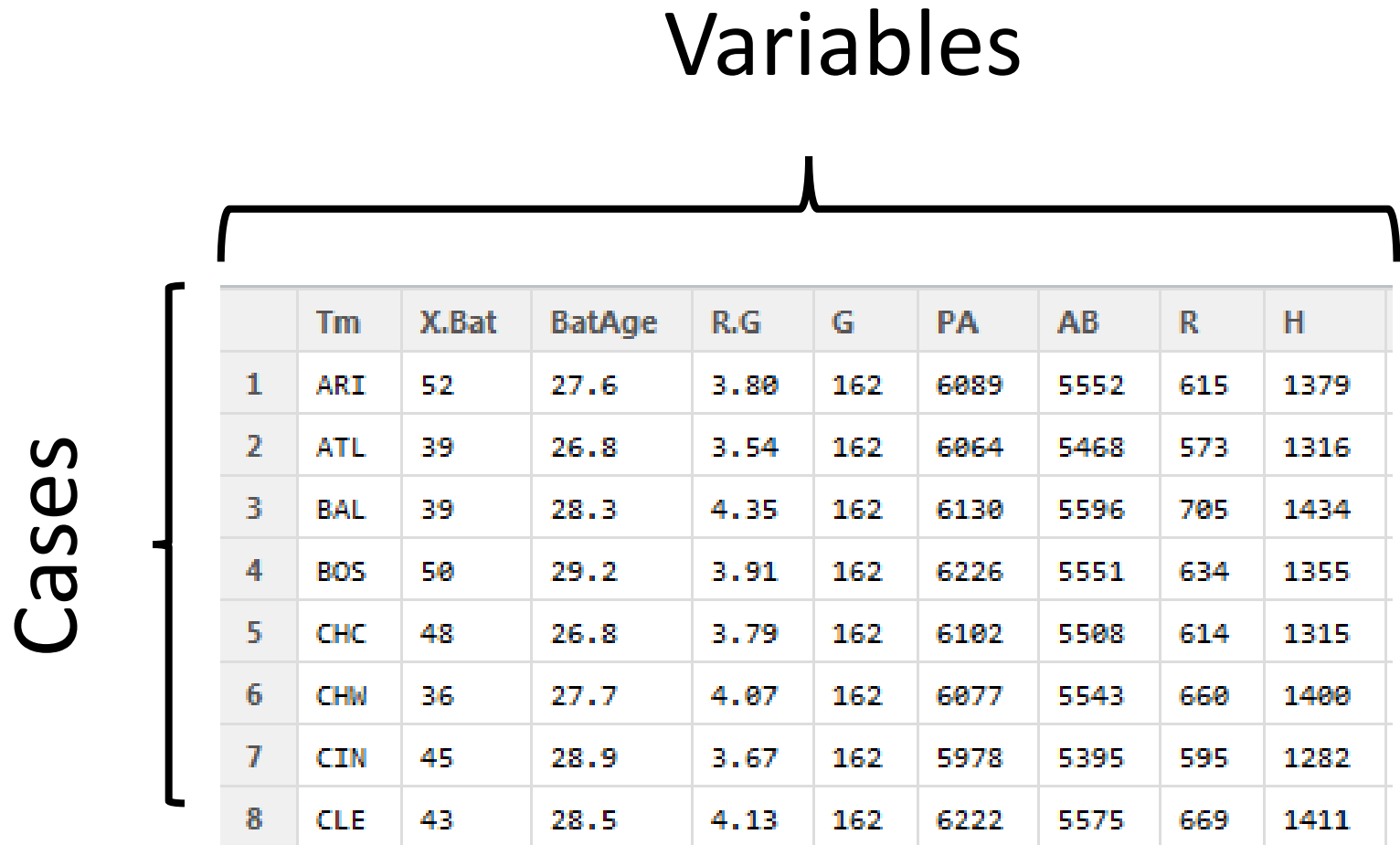


	playerID	nameFirst	nameLast	birthYear	birthMonth
1	aardsda01	David	Aardsma	1981	12
2	aaronha01	Hank	Aaron	1934	2
3	aaronto01	Tommie	Aaron	1939	8
4	aasedo01	Don	Aase	1954	9
5	abadan01	Andy	Abad	1972	8

Another Dataset: Team statistics

Variables

Cases



	Tm	X.Bat	BatAge	R.G	G	PA	AB	R	H
1	ARI	52	27.6	3.80	162	6089	5552	615	1379
2	ATL	39	26.8	3.54	162	6064	5468	573	1316
3	BAL	39	28.3	4.35	162	6130	5596	705	1434
4	BOS	50	29.2	3.91	162	6226	5551	634	1355
5	CHC	48	26.8	3.79	162	6102	5508	614	1315
6	CHW	36	27.7	4.07	162	6077	5543	660	1400
7	CIN	45	28.9	3.67	162	5978	5395	595	1282
8	CLE	43	28.5	4.13	162	6222	5575	669	1411

Loading the Lahman data into Python

We can use `pd.read_csv()` to read in .csv files as pandas DataFrames

```
import pandas as pd
```

```
batting = pd.read_csv('Batting.csv')
```

```
players = pd.read_csv("People.csv")
```

Pandas DataFrame operations

Selecting a subset of columns

1. We can select a subset of columns using:

```
my_df[["col1", "col2"]]
```

	playerID	yearID	stint	teamID	lgID	G
0	aardsda01	2004	1	SFN	NL	11
1	aardsda01	2006	1	CHN	NL	45
2	aardsda01	2007	1	CHA	AL	25
3	aardsda01	2008	1	BOS	AL	47

2. We can select a single column as a **Series** using:

```
my_df["col1"]
```

Useful to get a more manageable dataset

We can then apply numpy operations on the data.
E.g.,

```
total_stat = np.sum(my_df["col1"])
```

```
derived_stat = my_df["col1"] + my_df["col1"]
```

Useful to add new columns to a dataset

Let's explore these concepts in Jupyter!

Selecting a subset of rows based on position

We can select a subset of rows based on position using `.iloc`:

```
my_df.iloc[[0, 10, 20]]    # since given a list of indices, returns a DataFrame
```

This is often useful for getting the first or last row:

```
my_df.iloc[0]             # since given number as an index, returns a Series  
my_df.iloc[-1]            # last row
```

We can also select rows based on Index values using `.loc`

```
batting2 = batting.set_index()
```

Let's explore these concepts in Jupyter!

Sorting rows of a DataFrame

We can sort the rows of a DataFrame using:

```
my_df.sort_values("col_name")
```

We can sort the rows from largest to smallest value using:

```
my_df.sort_values("col_name", ascending = False)
```

Let's explore these concepts in Jupyter!

Getting a subset of rows using Boolean indexing

We can get a subset of rows using that meet particular criteria using Boolean indexing:

```
my_df[my_df["col_name"] == 7] # getting rows using a Boolean mask
```

We can also do this (in many cases) using the .query() method

```
my_df.query("col_name == 7") # getting rows using the query function
```

Let's explore these concepts in Jupyter!

Adding and renaming columns

We can add an ndarray (of the appropriate length) as a column to a DataFrame using:

```
my_df["new_col"] = col_values
```

We can rename columns using the .rename() method:

```
rename_dictionary = {"old_col_name": "new_col_name"}  
my_df.rename(columns = rename_dictionary )
```

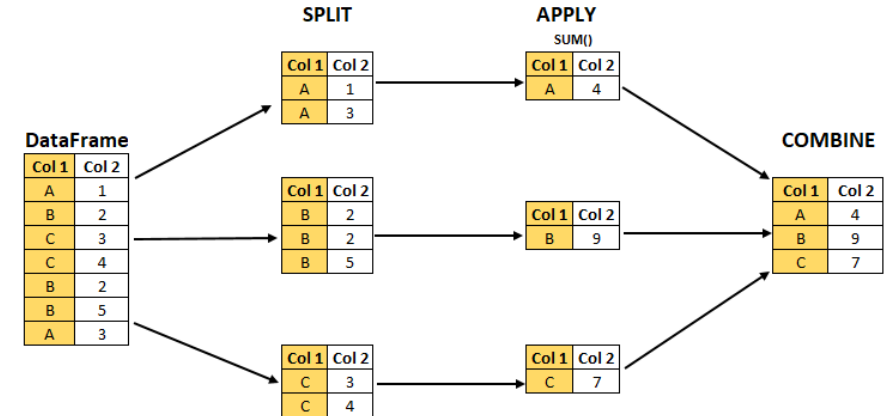
Let's explore these concepts in Jupyter!

Creating aggregate statistics by group

We can get statistics separately by group:

```
baseball.groupby("Year").agg("sum")
```

```
my_df.groupby("group_col_name").agg(  
    new_col1 = ('col_name', 'statistic_name1'),  
    new_col2 = ('col_name', 'statistic_name2'),  
    new_col3 = ('col_name', 'statistic_name3')  
)
```



```
batting.groupby("teamID").agg(  
    max_HR = ("HR", "max"),  
    first_player = ("playerID", "min")  
)
```

Let's explore these concepts in Jupyter!

Merging/Joining DataFrames

The DataFrames in the Lahman database can be connected through different ID columns

- For example, we can relate the Batting and People DataFrames using the playerID column

Batting DataFrame

	playerID	yearID	stint	teamID	lgID	G	G_batting	AB	R	H
1	aardsda01	2004	1	SFN	NL	11	11	0	0	0
2	aardsda01	2006	1	CHN	NL	45	43	2	0	0
3	aardsda01	2007	1	CHA	AL	25	2	0	0	0
4	aardsda01	2008	1	BOS	AL	47	5	1	0	0
5	aardsda01	2009	1	SEA	AL	73	3	0	0	0
6	aardsda01	2010	1	SEA	AL	53	4	0	0	0
7	aardsda01	2012	1	NYA	AL	1	NA	NA	NA	NA
8	aaronha01	1954	1	ML1	NL	122	122	468	58	131

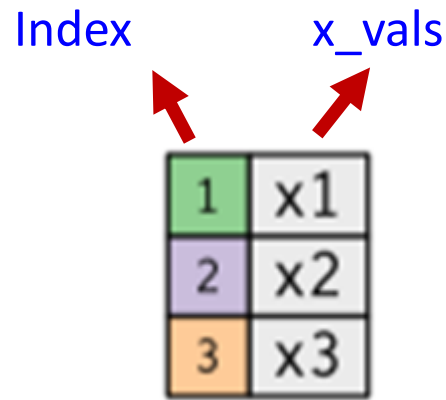
People DataFrame

	playerID	nameFirst	nameLast	birthYear	birthMonth
1	aardsda01	David	Aardsma	1981	12
2	aaronha01	Hank	Aaron	1934	2
3	aaronto01	Tommie	Aaron	1939	8
4	aasedo01	Don	Aase	1954	9
5	abadan01	Andy	Abad	1972	8

Joining

Suppose we have two DataFrames (or Series) called **x_df** and **y_df**

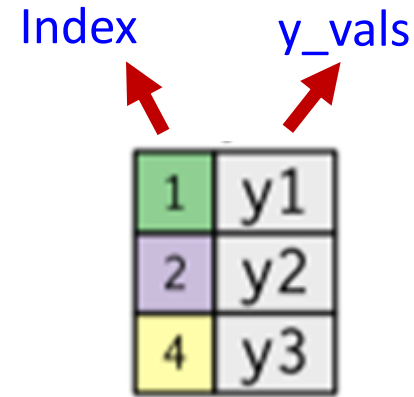
- **x_df** have one column called **x_vals**
- **y_df** has one column called **y_vals**



A 3x2 grid representing a DataFrame. The first column contains index values 1, 2, and 3, each in a colored box (green, purple, orange). The second column contains values x1, x2, and x3. Red arrows point from the labels 'Index' and 'x_vals' to the respective columns.

1	x1
2	x2
3	x3

DataFrame: x_df



A 3x2 grid representing a DataFrame. The first column contains index values 1, 2, and 4, each in a colored box (green, purple, yellow). The second column contains values y1, y2, and y3. Red arrows point from the labels 'Index' and 'y_vals' to the respective columns.

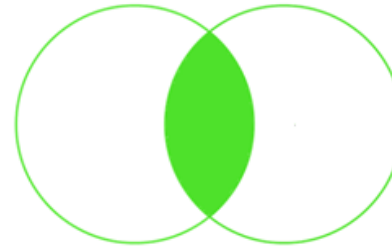
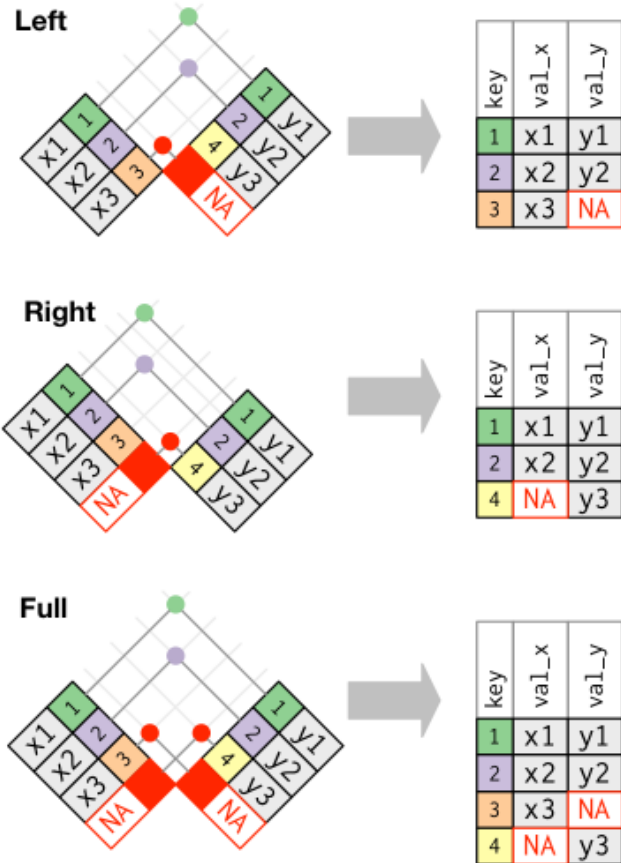
1	y1
2	y2
4	y3

DataFrame: y_df

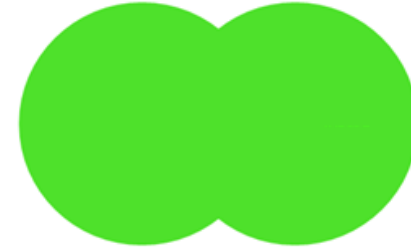
We can join these two DataFrames into a single DataFrame by aligning rows with the same Index value using the general syntax: **x_df.join(y_df)**

- i.e., the new joined data frame will have two columns: **x_vals**, and **y_vals**

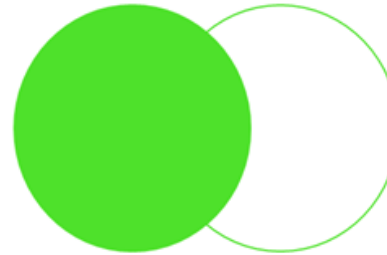
Joining



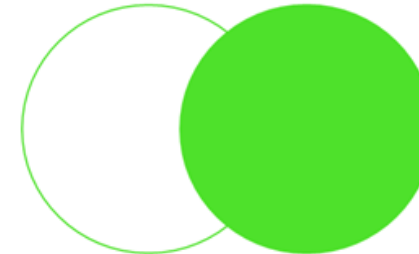
INNER JOIN



FULL OUTER JOIN



LEFT JOIN



RIGHT JOIN

`x_df.join(y_df, how = "left")` # based on Index

`x_df.merge(y_df, how = "inner", left_on = "x_col", right_on = "y_col")` # based on columns

If there is time...

What are “good” statistics?

How could we determine what a “good” value for a statistic is?

- i.e., how many home runs would need to be hit to determine if a player is “good at hitting home runs”?

One method to determine what a “good” statistic is, would be a value that is say greater than 90% of baseball players

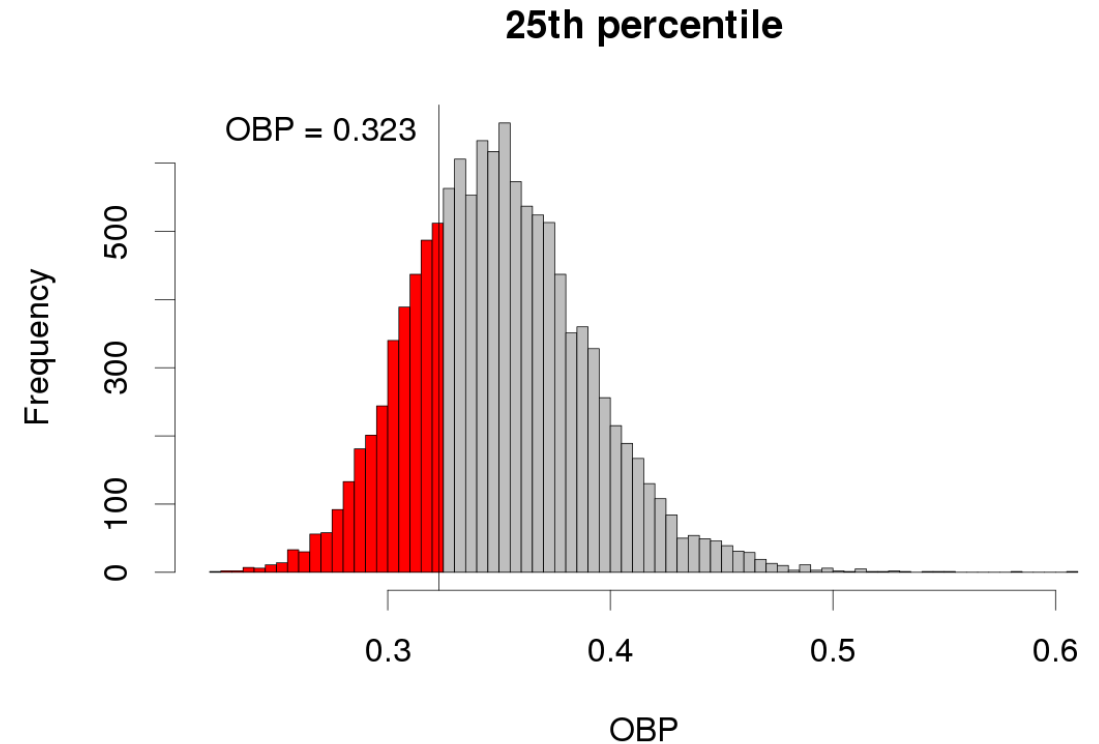
Percentiles

The **p^{th} percentile** is the value **v** of a quantitative variable such that p percent of the data is less than **v**

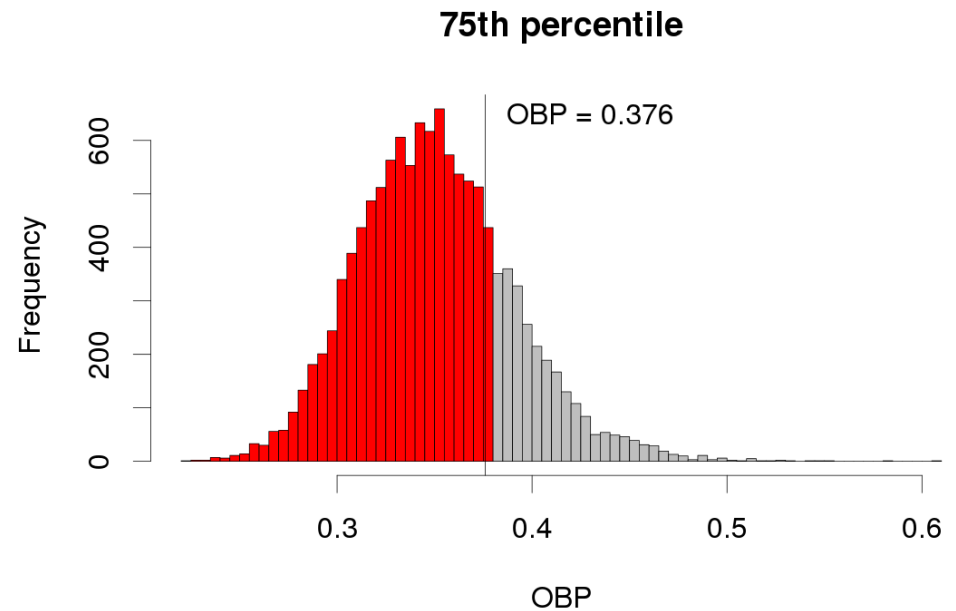
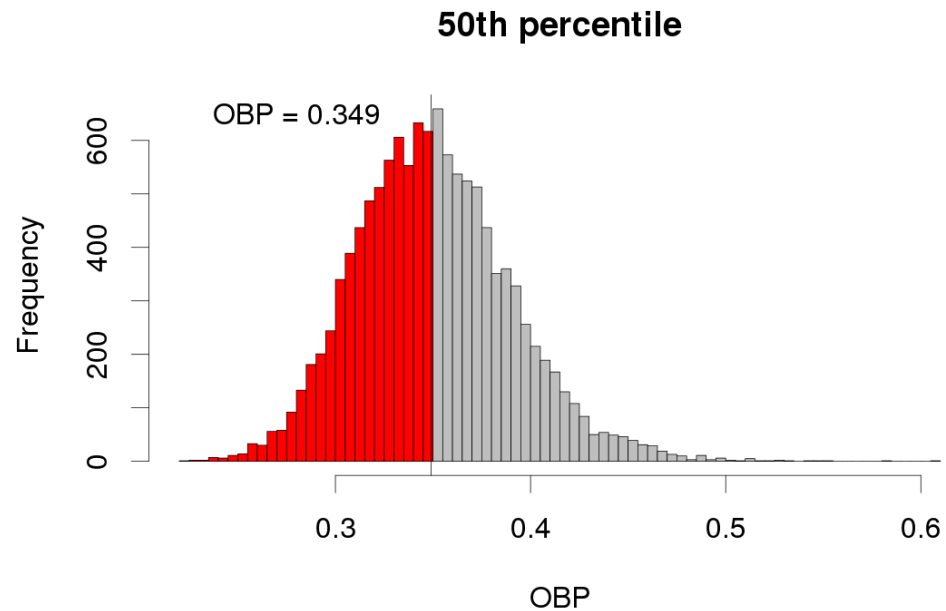
We can use the `my_df.quantile()` get a percentile value:

get the 90th percentile

```
my_df.quantile(.9)
```



Percentiles/quantiles



https://emeyers.shinyapps.io/baseball_stat_percentiles/

What is a good statistic for...?

What are “good” values are for the following statistics:

- I used the years from 1971 to 2014, min PA = 500

Home runs (HR)?

- See class activities

Batting average (BA)

- .313

On base percentage (OBP)?

- .394

Strikeouts (SO)

- 47
- Bad is 129 (90th percentile)

Five Number Summary

Five Number Summary = (min, Q_1 , median, Q_3 , max)

Q_1 = 25th percentile (also called 1st quartile)

Q_3 = 75th percentile (also called 3rd quartile)

Roughly divides the data into fourths

Range and Interquartile Range

Range = maximum – minimum

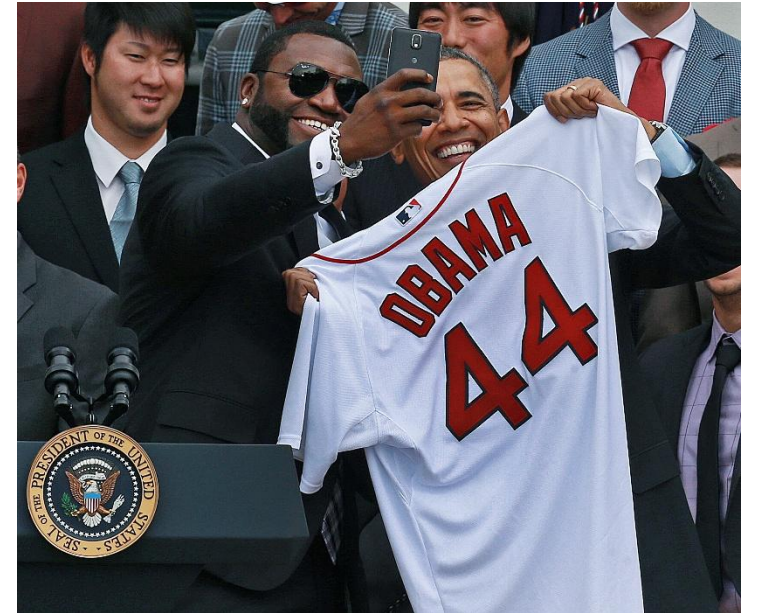
Interquartile range (IQR) = $Q_3 - Q_1$

Detecting of outliers

As a rule of thumb, we call a data value an **outlier** if it is:

Smaller than: $Q_1 - 1.5 * IQR$

Larger than: $Q_3 + 1.5 * IQR$



Are there any outlier years in David Ortiz home run numbers?

1. Five Number Summary: (23, 28.5, 32, 36, 54)

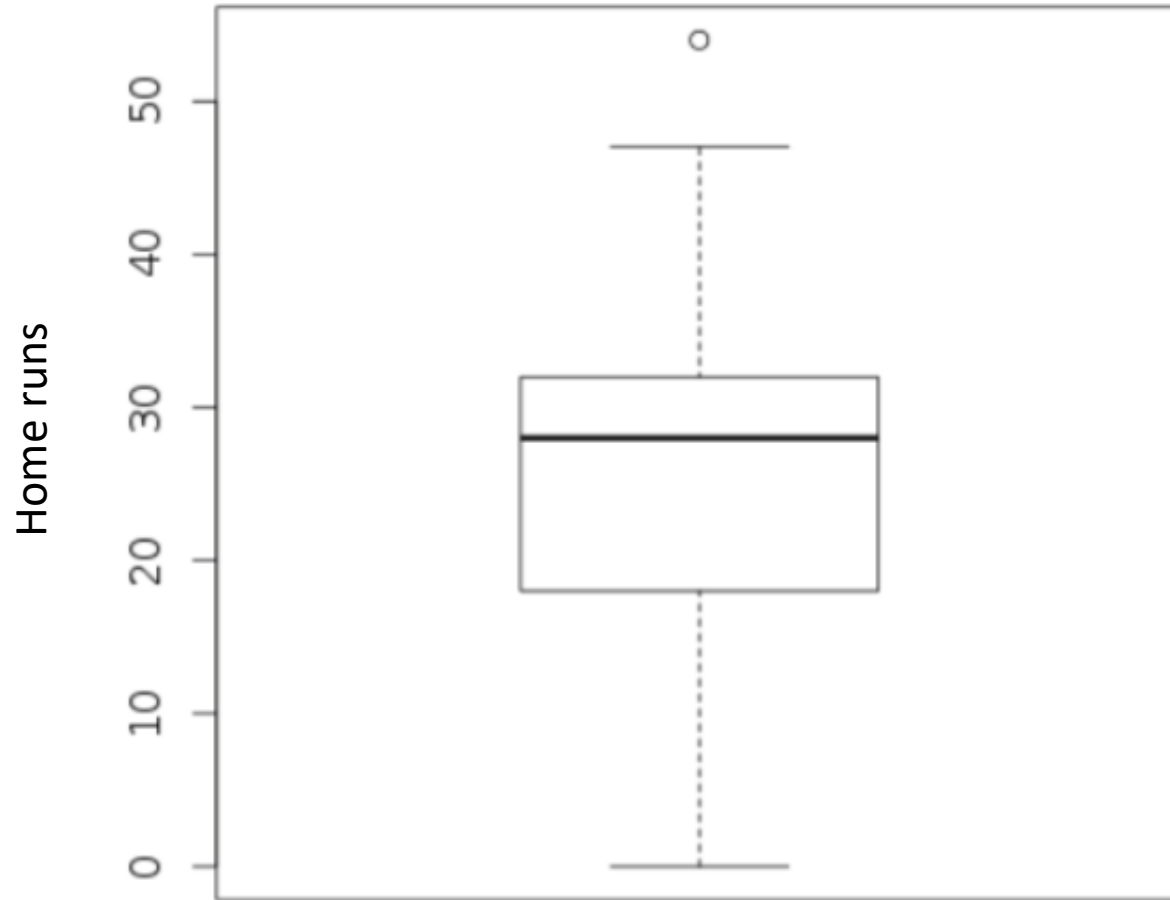
2. Interquartile range (IQR) = 7.5

Boxplots

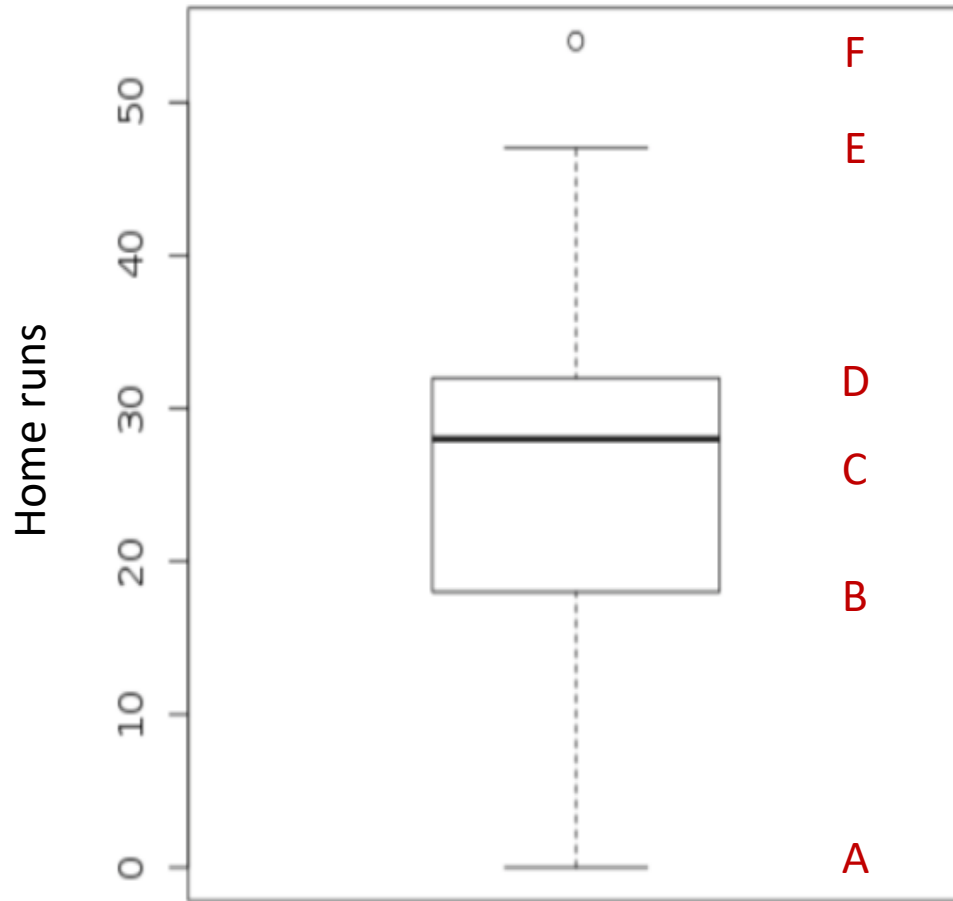
A **boxplot** is a graphical display of the 5 number summary and consists of:

1. Drawing a box from Q_1 to Q_3
2. Dividing the box with a line drawn at the median
3. Draw a line from each quartile to the most extreme data value that is not and outlier
4. Draw a dot/asterisk for each outlier data point.

Box plot of David Ortiz home runs



Box plot quiz



What is:

- Q1?
- Q3?
- The median?
- Most extreme values that are not outliers
- Outliers

Who is better?

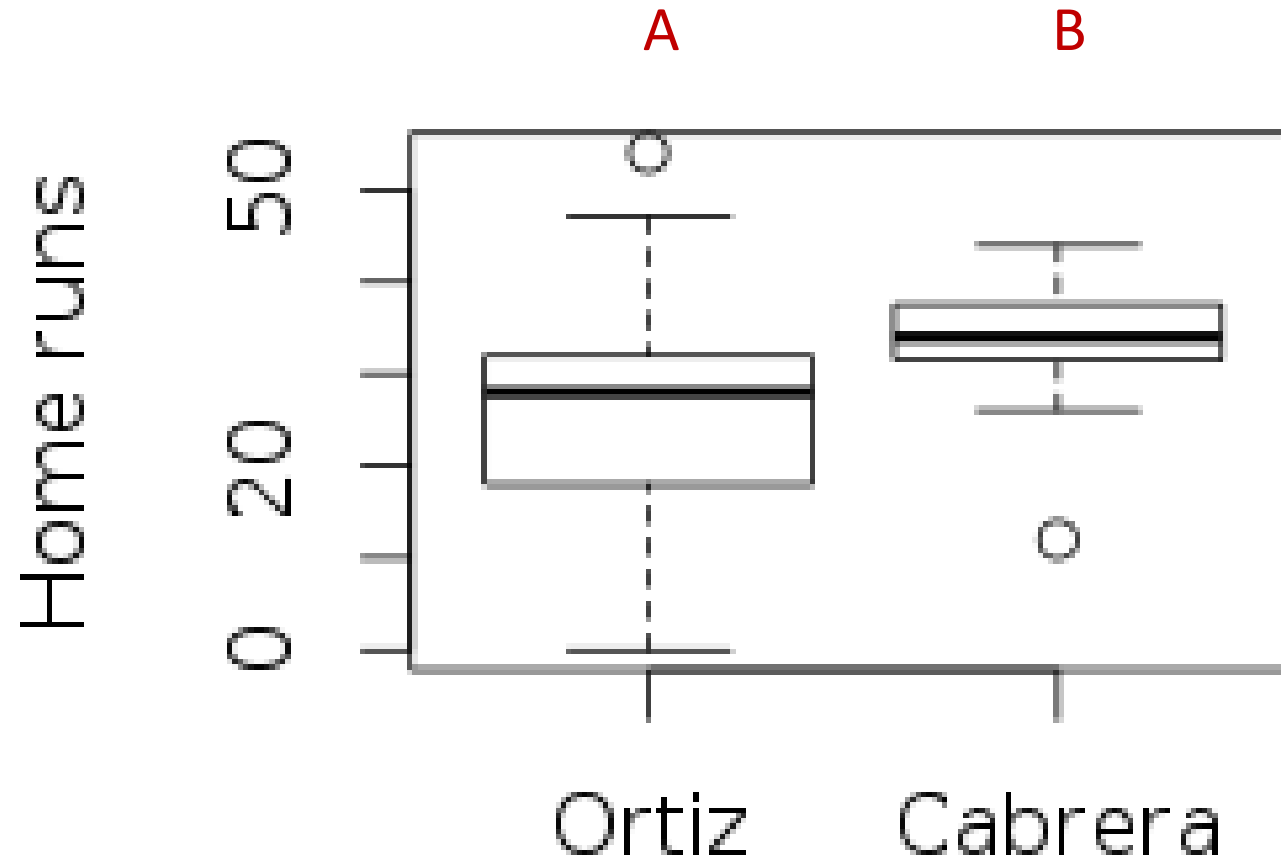


Miguel Cabrera:
HR in 2014 = 25



David Ortiz:
HR in 2014 = 35

Comparing players with side-by-side box plots



How would you describe the differences between these two players in terms of HRs?
Who is better?

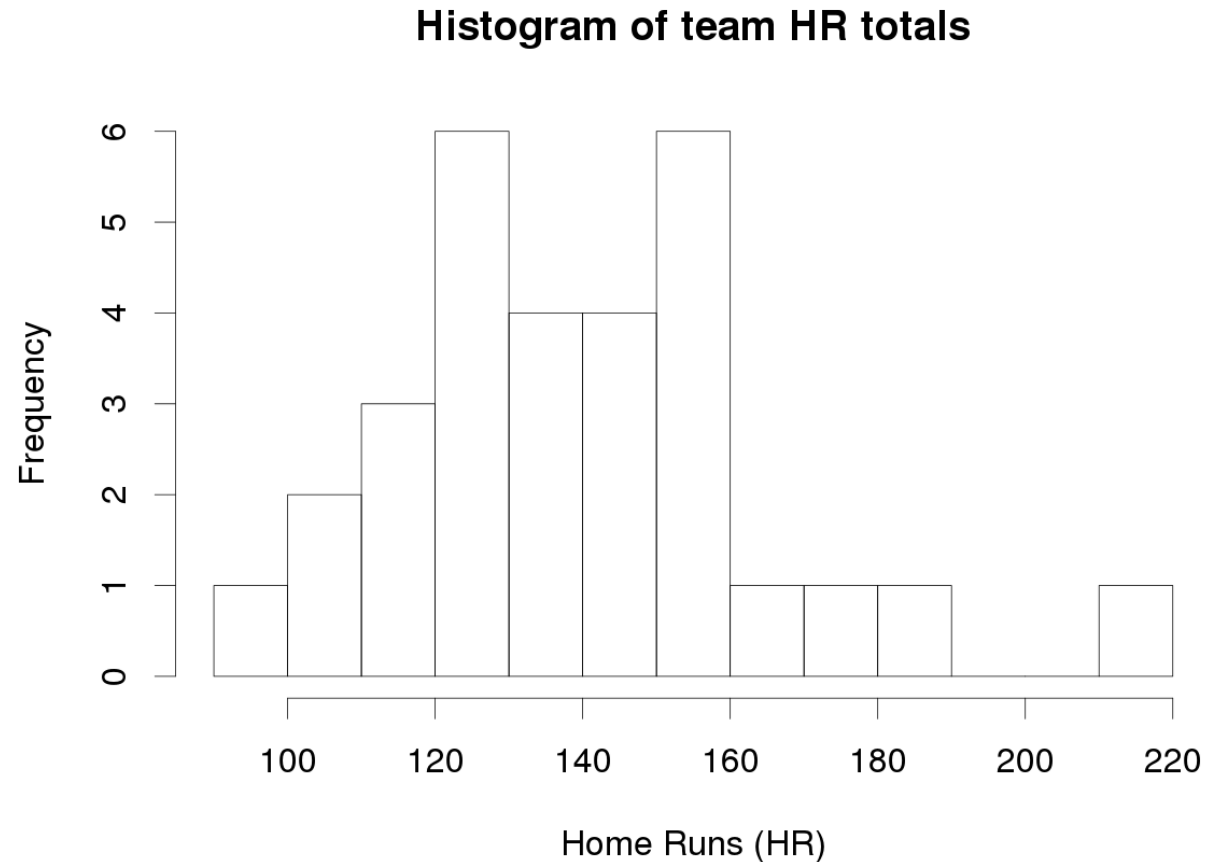
Visualizing the 'shape' of how data is distributed

Boxplots can give us a sense of some key statistics about our data

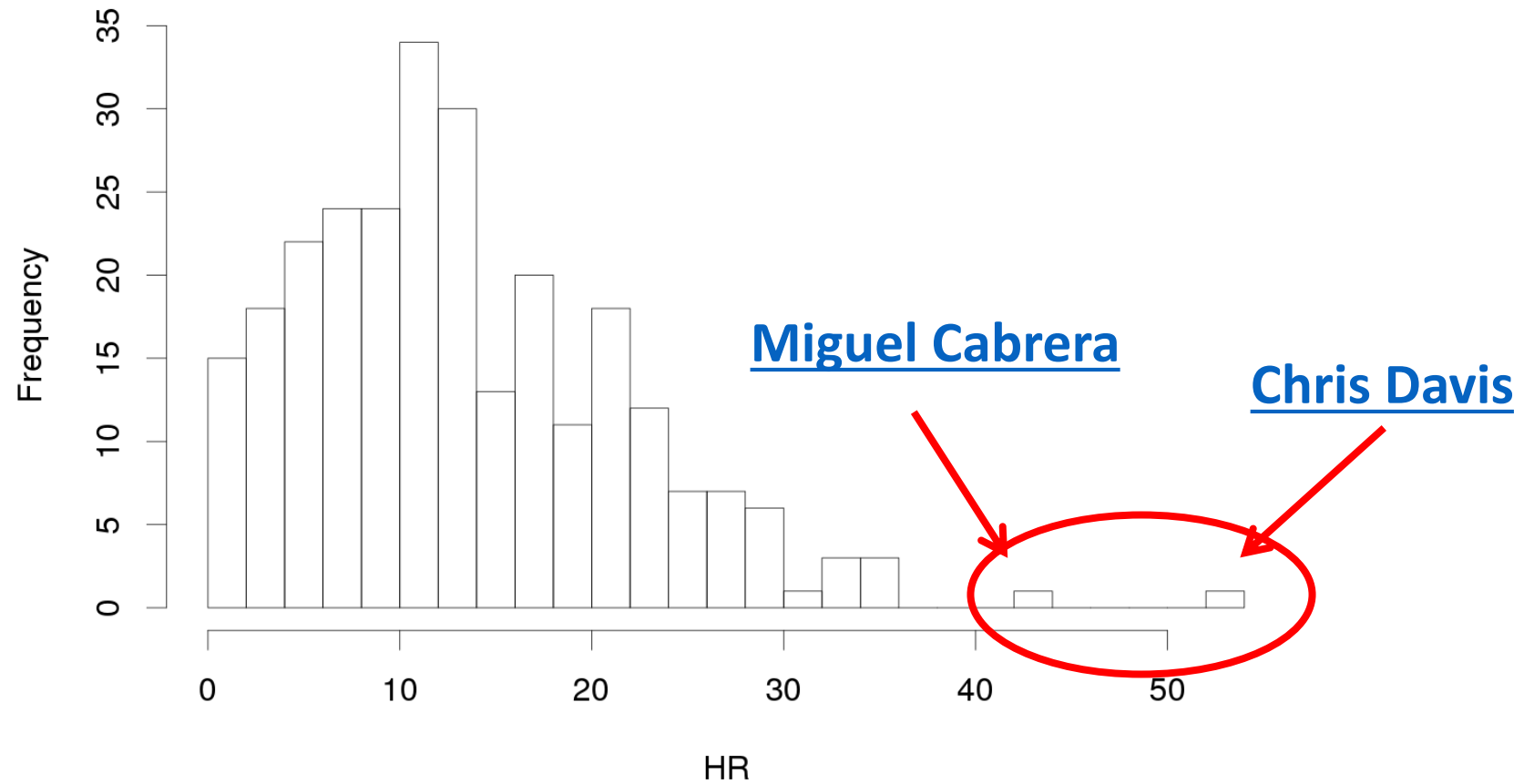
There are other methods that can give us a better picture of the shape of how all the data is distributed

Histograms

Another related way to get a sense of the shape of a distribution is to use a **histogram**

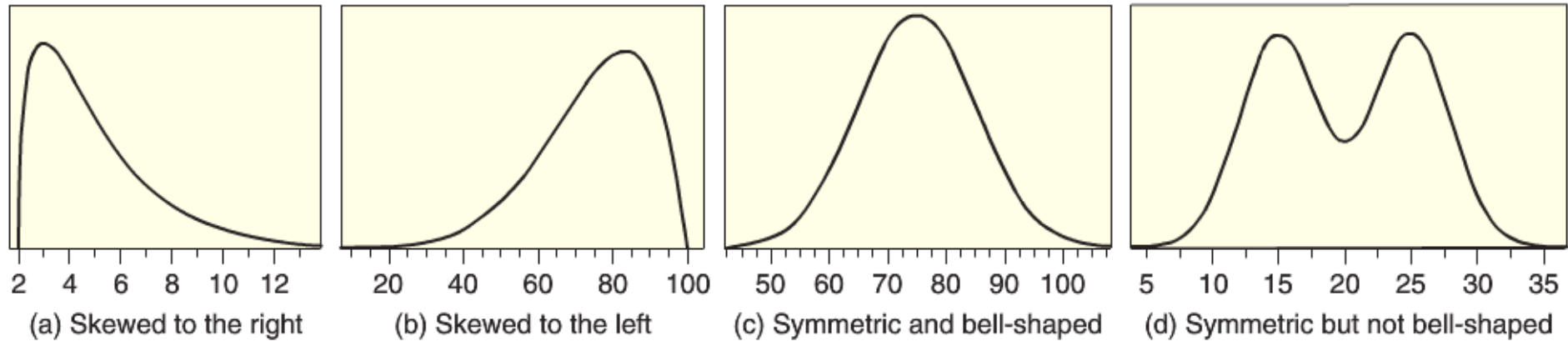


Histogram of HRs for 2013 players with over 300 PA



Observations about the distribution?

Common shapes for Distributions



Let's explore these concepts in Jupyter!

For next class

1. Read chapter 2 of Astrobball
2. Complete the labs:
 - Lab 1: due 11pm on Wednesday
 - Lab 2: due 11pm on Sunday

